IoT Challenge 3

Node-Red Report

Marco Abbadessa - 10854816
Alice Berta - 10889590 - leader

Academic Year 2025–2026

Contents

1 Introduction **2**

2 Flow Overview **2**

3 Task 1 – Publisher Flow (1 point) **2**
3.1 Description . . . 2
3.2 Implementation . . . 3
3.3 Output: `id_log.csv` . . . 4

4 Task 2 – Subscriber Branch with ZigBee Processing (2 points) **4**
4.1 Description . . . 4
4.2 Implementation . . . 4
4.3 MQTT Publish Payload Format . . . 10
4.4 Output: `filtered_elems.csv` . . . 10
4.5 Node-Red Dashboard Charts . . . 10

5 Task 3 – Link Status Handling and ThingSpeak Publishing (2 points) **11**
5.1 Description . . . 11
5.2 Implementation . . . 11
5.3 Output: `outgoing_cost.csv` . . . 15
5.4 ThingSpeak Channel . . . 15

1 Introduction

The goal of this challenge is to design and implement a Node-Red flow that integrates MQTT messaging with ZigBee packet processing and external data publishing. The flow is organized around a single pipeline that handles two distinct roles: a publisher that generates periodic MQTT messages, and a subscriber branch that reacts to those messages by performing CSV lookups and conditional routing logic.

The challenge is divided into three progressive tasks, each building on the previous one. Task 1 sets up the publisher side of the flow. Task 2 adds the subscriber branch with CSV-based lookup, conditional MQTT publishing, attribute filtering, and Node-Red dashboard charting. Task 3 extends the subscriber branch further by handling Link Status packets and sending the extracted data to a public ThingSpeak channel.

2 Flow Overview

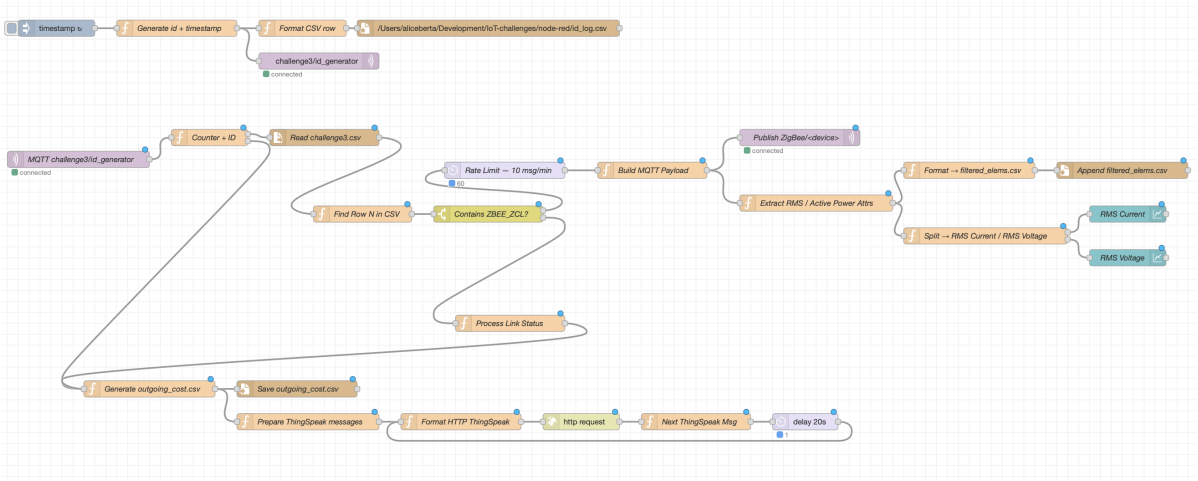


Figure 1: *Complete flow.*

The overall flow is composed of a single tab containing two main branches sharing the same MQTT broker (`localhost`, port `1884`):

- **Publisher branch**: a periodic trigger generates a random ID and a UNIX timestamp, packages them as a JSON payload, publishes the message to the topic `challenge3/id_generator`, and simultaneously writes the data to `id_log.csv`.
- **Subscriber branch**: an MQTT-in node subscribes to the same topic, applies modulo arithmetic on the received ID to compute a packet index `N`, reads the corresponding row from `challenge3.csv`, and dispatches the processing to one of three conditional paths depending on the packet type.

3 Task 1 – Publisher Flow (1 point)

3.1 Description

The first task requires building a flow that periodically publishes an MQTT message to the local Mosquitto broker at `localhost:1884`, on the topic `challenge3/id_generator`. Each message must carry a JSON payload containing two fields: a random integer `id` drawn uniformly in the

range $[0, 30000]$, and a `timestamp` field representing the current UNIX time at the moment of generation. The publishing rate is fixed at one message per second. Additionally, every sent message must be logged to a CSV file named `id_log.csv`, structured as `No.,ID,TIMESTAMP`, where `No.` is an incremental row counter.

3.2 Implementation

The publisher branch of the Node-RED flow was constructed to periodically generate data, publish it via MQTT, and log the records locally.

The first node is an **Inject node** called "1s Trigger". It acts as the primary clock for the flow, configured to fire automatically at a regular interval of 1 second.

The second node is a **Function node** called "Generate ID + Timestamp". Connected to the Inject node, it generates a random integer ID between 0 and 30,000 alongside the current UNIX timestamp. It then converts this object into a JSON-formatted string payload to ensure proper compatibility with the MQTT broker. The internal logic is implemented as follows:

Listing 1: *Generate ID + Timestamp logic*

```
msg.payload = {
    id: Math.floor(Math.random() * 30001),
    timestamp: Math.floor(Date.now() / 1000)
};
msg.payload = JSON.stringify(msg.payload);
return msg;
```

The third node is an **MQTT Out** node. Receiving the JSON string from the previous function, this node connects to the local Mosquitto broker at `localhost:1884` and publishes the payload to the `challenge3/id_generator` topic.

The fourth node is a **Function node** called "Format CSV row". Branching off the data generation node, this block parses the JSON payload and uses a persistent context variable to maintain an incremental row counter. It formats the extracted data into a comma-separated string, adding the required header specifically for the first row. The code used is shown below:

Listing 2: *Format CSV row logic*

```
let count = context.get('count') || 0;
count++;
context.set('count', count);

const data = JSON.parse(msg.payload);

if (count === 1) {
    msg.payload = "No.,ID,TIMESTAMP\n" +
                  `${count},${data.id},${data.timestamp}\n`;
} else {
    msg.payload = `${count},${data.id},${data.timestamp}\n`;
}
return msg;
```

The fifth node is a **File node** called "id_log.csv". Finally, this node receives the formatted CSV string and appends it to the local log file. The automatic newline feature is intentionally disabled to maintain the precise line break formatting established by the preceding function node.

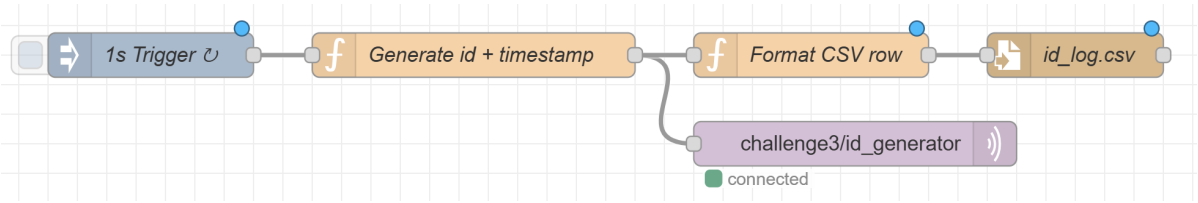


Figure 2: *Publisher flow.*

3.3 Output: `id_log.csv`

Listing 3: *Excerpt of generated* `id_log.csv`.

```
No.,ID,TIMESTAMP
1,16320,1778573262
2,5225,1778573263
3,13278,1778573264
4,23212,1778573265
```

4 Task 2 – Subscriber Branch with ZigBee Processing (2 points)

4.1 Description

The subscriber branch listens on the same `challenge3/id_generator` topic. Each time a message arrives, the branch extracts the `id` field and uses it to compute a row index:

$$N = \texttt{id} \bmod 5218$$

The row at position `N` in `challenge3.csv` is then fetched and inspected.

If its `Command String` column contains a `ZBEE_ZCL` layer, the packet is published to a topic named after the ZigBee source device. The publish rate is capped at 10 messages per minute. When the published packet also carries `RMS Current`, `RMS Voltage`, or `Active Power` attributes, those values are extracted and appended to `filtered_elems.csv`; the two electrical measurements are also plotted in real time on the Node-Red dashboard.

Packets that do not contain `ZBEE_ZCL` are silently discarded. The branch stops after exactly 200 messages have been received, whether or not each message produced a publishable packet.

4.2 Implementation

The subscriber branch is a linear chain of function and utility nodes. Each node passes its result forward; returning `null` at any point stops that message immediately.

During development, the external MQTT publisher was not yet available. To allow independent testing, the MQTT-in node was temporarily replaced by an **Inject node** configured to fire every second. The random `id` and UNIX timestamp were generated inside the *Counter + ID* function node itself. Switching to the real MQTT subscriber requires only reconnecting the wire from the Inject node to the MQTT-in node; no other change to the flow is needed.

Node 1 — Counter + ID

This function node is the entry point of the subscriber branch and has two outputs. Its first job is to maintain a running count of messages processed so far, stored in `flow.get('msgCount')`. If the count has already exceeded 200, the node logs a warning and returns `null`, dropping the message silently.

Listing 4: *Counter + ID – function node.*

```
var count = flow.get('msgCount') || 0;
count++;
if (count > 200) {
    node.warn('200-message limit reached -- stopping.');
    return null;
}
flow.set('msgCount', count);

var id, ts;
if (msg.payload && typeof msg.payload === 'object' &&
        msg.payload.id !== undefined) {
    id = Number(msg.payload.id);
    ts = Number(msg.payload.timestamp);
} else {
    id = Math.floor(Math.random() * 30001);
    ts = Math.floor(Date.now() / 1000);
}

msg.subId = id;
msg.subTs = ts;
msg.N = id % 5218;
msg.msgCount = count;
msg.filename = flow.get('csvPath') ||
    '/Users/aliceberta/Development/IoT-challenges/node-red/challenge3.csv';

if (count === 200) {
    node.send([msg, { payload: 'save' }]);
    return null;
}
return msg;
```

If the limit has not been reached, the node reads `id` and `timestamp` from the JSON payload delivered by the MQTT broker. The row index `N` is computed as `id mod 5218` and the path to the CSV file is attached to the message. The current count is also stored in `msg.msgCount` for reference by downstream nodes. When the count reaches exactly 200, the node fires a trigger message on its second output (`{payload: 'save'}`) to initiate the final dump of `outgoing_cost.csv`, then stops processing by returning `null`.

Node 2 — Read challenge3.csv

A **File In** node reads the entire content of `challenge3.csv` as a UTF-8 string into `msg.payload`. The file path is taken dynamically from `msg.filename`, so no path is hard-coded inside the node configuration. All properties set by the previous node (`msg.N`, `msg.subId`, etc.) are preserved across the read.

Node 3 — Find Row N

The built-in CSV node is not used here because the `Command String` column contains commas inside quoted substrings, which would break a simple split. A custom character-by-character parser handles quoted fields correctly.

Listing 5: *Find Row N – function node.*

```
var N = msg.N;
var csvText = msg.payload;
```

```
function parseLine(line) {
    var res = [], field = '', inQ = false;
    for (var i = 0; i < line.length; i++) {
        var c = line[i];
        if (c === '"') { inQ = !inQ; }
        else if (c === ',' && !inQ) { res.push(field.trim()); field = ''; }
        else { field += c; }
    }
    res.push(field.trim());
    return res;
}

var lines = csvText.split('\n');
var headers = parseLine(lines[0]);

for (var i = 1; i < lines.length; i++) {
    if (!lines[i].trim()) continue;
    var fields = parseLine(lines[i]);
    if (parseInt(fields[0]) === N) {
        var row = {};
        headers.forEach(function(h, j) { row[h] = fields[j] || ''; });
        msg.row = row;
        msg.commandStr = row['Command String'] || '';
        return msg;
    }
}
node.warn('Packet N=' + N + ' not found in CSV -- skipping');
return null;
```

The parser flips a boolean flag `inQ` each time it meets a double-quote character, and only splits on commas found while `inQ` is `false`. Once the matching row is located, its fields are stored as a key-value object in `msg.row` and the command string is copied into `msg.commandStr` for downstream nodes. If no row matches, a warning is logged and the message is dropped.

Node 4 — ZBEE_ZCL?

A **Switch** node checks whether `msg.commandStr` contains the substring `ZBEE_ZCL`. Only messages that satisfy this condition are forwarded; all others are silently dropped. This single check filters out Link Status frames and any other protocol that is not a ZigBee Cluster Library packet.

Node 5 — Max 10 msg/min

A **Delay** node configured in *Rate Limit* mode ensures that no more than 10 messages per minute are forwarded to the publish stage. Messages that arrive faster than this rate are queued and released gradually; none are dropped.

Node 6 — Build MQTT Payload

This function node assembles the JSON payload required by the specification and sets the dynamic MQTT topic.

Listing 6: *Build MQTT payload – function node.*

```
var row = msg.row;
var deviceName = (row['Device Name ZigBee Source'] || 'unknown')
```

```
    .trim()
    .replace(/[^a-zA-Z0-9\s\-_]/g, '')
    .trim() || 'unknown';

var payload = {
    timestamp : Math.floor(Date.now() / 1000),
    id : msg.subId,
    'wpan.src' : row['Source Address'] || '',
    'wpan.dst' : row['Destination Address'] || '',
    'zbee.src' : row['Source Address ZigBee'] || '',
    'zbee.dst' : row['Destination Address ZigBee'] || '',
    topic : 'ZigBee/' + deviceName,
    payload : msg.commandStr
};

msg.topic = 'ZigBee/' + deviceName;
msg.payload = JSON.stringify(payload);
msg.pubTs = payload.timestamp;
msg.deviceName = deviceName;
return msg;
```

The device name is read from the `Device Name ZigBee Source` column and sanitised with a regular expression that removes any character that would be invalid in an MQTT topic path. The assembled payload is serialised to JSON and placed in `msg.payload`; the topic string is stored in `msg.topic` so that the MQTT Out node can pick it up dynamically. The publish timestamp is also saved as `msg.pubTs` for use by the CSV writer downstream.

From the output of *Build MQTT Payload* two wires depart in parallel. The first connects to an **MQTT Out** node that publishes the assembled payload to `localhost:1884` on the dynamic topic stored in `msg.topic`. The second wire leads to the *Extract RMS / Active Power Attrs* function node.

Node 7 — Extract RMS / Active Power Attrs

The `Command String` field is formatted as a Python dictionary with single-quoted keys and values. The `parsePyDict` helper converts it to valid JSON before parsing.

Listing 7: *Extract RMS / Active Power Attrs – function node.*

```
var cmdStr = msg.commandStr;

function parsePyDict(s) {
    s = s.replace(/'/g, '"');
    s = s.replace(/\bNone\b/g, 'null');
    s = s.replace(/\bTrue\b/g, 'true');
    s = s.replace(/\bFalse\b/g, 'false');
    try { return JSON.parse(s); } catch(e) {
        node.warn('Parse error: ' + e.message);
        return null;
    }
}

var d = parsePyDict(cmdStr);
if (!d) { return null; }
if (!d['Status']){ return null; }

function toArr(v) { return v === undefined ? [] : [].concat(v); }
```

```
var attrs = toArr(d['Attribute']);
var stats = toArr(d['Status']);
var dtypes = toArr(d['Data Type']);
var seqNum = d['Sequence Number'] || '';
var int16Arr = toArr(d['Int16']);
var uint16Arr = toArr(d['Uint16']);
var int16i = 0, uint16i = 0;

var TARGETS = ['RMS Current', 'RMS Voltage', 'Active Power'];
var found = [];

for (var i = 0; i < attrs.length; i++) {
    var attrName = (attrs[i] + '').split(' (')[0].trim();
    var statusStr = (stats[i] + '').split(' (')[0].trim();
    var dtypeStr = (dtypes[i] + '').split(' (')[0].trim();
    var dtype = dtypeStr.toLowerCase();
    var val = '';

    if (dtype.includes('signed') && !dtype.includes('unsigned'))
        val = (int16Arr[int16i++] + '').split(' (')[0].trim();
    else if (dtype.includes('unsigned'))
        val = (uint16Arr[uint16i++] + '').split(' (')[0].trim();

    if (TARGETS.some(function(t) { return attrName.includes(t); }))
        found.push({ attribute: attrName, status: statusStr,
                     dataType: dtypeStr, value: val, seqNum: seqNum });
}

if (found.length === 0) { return null; }
msg.filteredAttrs = found;
return msg;
```

Only *Read Attributes Response* packets contain both a `Status` and numeric value arrays; packets without a `Status` key are discarded immediately. The helper `toArr` wraps a single value in an array so that the loop works identically whether the CSV field holds one attribute or many. Because the CSV stores signed integers under the key `Int16` and unsigned integers under `Uint16`, the correct counter (`int16i` or `uint16i`) is incremented for every attribute visited, preserving positional alignment even for attributes that are not in the target list. Attribute names, status strings, and data type labels are truncated at the first space-parenthesis sequence (`split(' (')[0]`) to strip the numeric code appended by Wireshark.

From *Extract RMS / Active Power Attrs* two further wires depart in parallel: one leads to the CSV writer chain, the other to the chart splitter.

Node 8 — Format filtered_elems.csv

Listing 8: *Format filtered_elems.csv – function node.*

```
var ts = msg.pubTs || Math.floor(Date.now() / 1000);
var count = flow.get('filteredCount') || 0;
var lines = [];

msg.filteredAttrs.forEach(function(item) {
    count++;
    lines.push([count, ts, item.seqNum, item.attribute,
                item.status, item.dataType, item.value].join(','));
```

```
});
flow.set('filteredCount', count);

var out = '';
if (!flow.get('filteredHeader')) {
    out = 'No.,Timestamp,Sequence Number,Attribute,' +
          'Status,Data Type,Data Value\n';
    flow.set('filteredHeader', true);
}
out += lines.join('\n') + '\n';

msg.payload = out;
return msg;
```

A flow-scoped counter keeps the row numbers consistent across multiple messages. The CSV header is written exactly once, the first time this node executes, controlled by the boolean flag `filteredHeader`. The output is then appended to `filtered_elems.csv` by a **File Out** node configured in append mode with automatic newlines disabled.

Node 9 — Chart Split

Listing 9: *Chart Split – function node (2 outputs).*

```
var currentMsg = null;
var voltageMsg = null;

msg.filteredAttrs.forEach(function(item) {
    var numVal = parseFloat(item.value);
    if (isNaN(numVal)) numVal = 0;

    if (item.attribute.includes('RMS Current'))
        currentMsg = { payload: numVal, topic: 'RMS Current' };
    else if (item.attribute.includes('RMS Voltage'))
        voltageMsg = { payload: numVal, topic: 'RMS Voltage' };
});

return [currentMsg, voltageMsg];
```

This two-output function node separates RMS Current and RMS Voltage into independent messages. Output 0 feeds the RMS Current chart and output 1 feeds the RMS Voltage chart on the Node-Red dashboard. Active Power is not plotted and is therefore ignored here. If a particular attribute is absent from the current message, the corresponding output remains `null` and no message is sent to that chart.

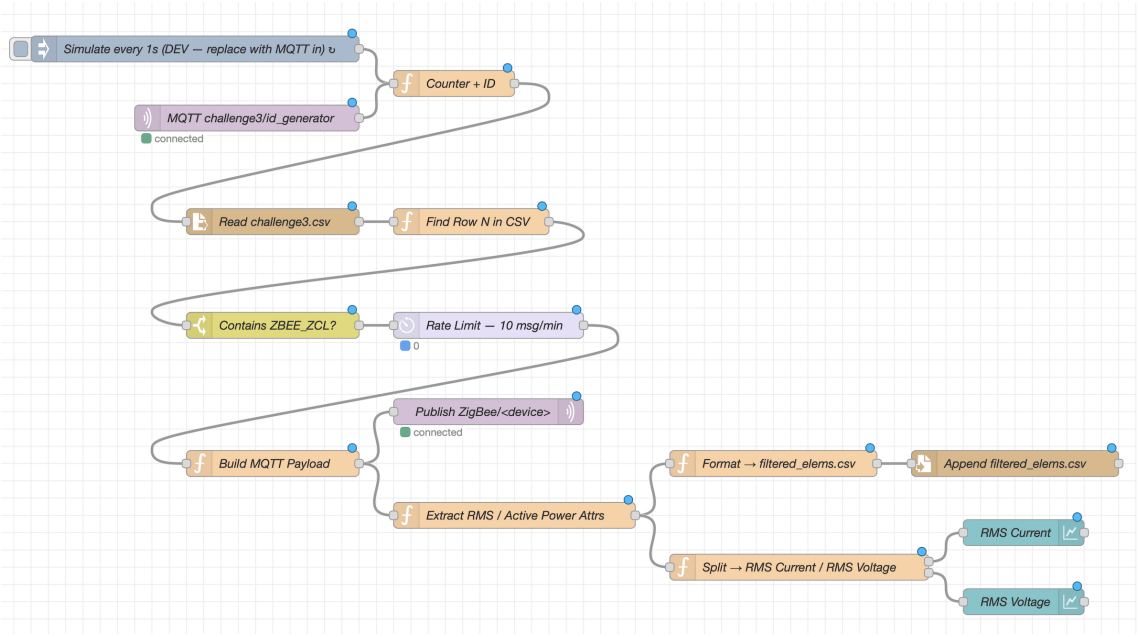


Figure 3: *Subscriber flow.*

4.3 MQTT Publish Payload Format

When a `ZBEE_ZCL` layer is detected, the message published to `ZigBee/<ZB_SRC_DEV_NAME>` uses the following JSON structure:

```
{
  "timestamp": CURRENT_TIMESTAMP,
  "id": SUB_ID,
  "wpan.src": "SRC_ADDR",
  "wpan.dst": "DST_ADDR",
  "zbee.src": "ZB_SRC_ADDR",
  "zbee.dst": "ZB_DST_ADDR",
  "topic": "ZigBee/ZB_SRC_DEV_NAME",
  "payload": CMD_STR
}
```

All address fields are in hexadecimal format (e.g., `0x1cd8`). `SUB_ID` is the original `id` value received from the subscription (before the modulo), and `CURRENT_TIMESTAMP` is the UNIX time at the moment the publish message is sent.

4.4 Output: `filtered_elems.csv`

Listing 10: *Excerpt of generated* `filtered_elems.csv`.

```
No.,Timestamp,Sequence Number,Attribute,Status,Data Type,Data Value
1,1778573280,245,Active Power,Success,16-Bit Signed Integer,0
2,1778573280,245,RMS Current,Success,16-Bit Unsigned Integer,0
3,1778573280,245,RMS Voltage,Success,16-Bit Unsigned Integer,219
```

4.5 Node-Red Dashboard Charts

The RMS Current and RMS Voltage values are displayed in real time on two separate line charts on the Node-Red dashboard, accessible at `http://localhost:1880/ui`. The dashboard was

built using the `@flowfuse/node-red-dashboard` package. Both charts belong to the group *Electrical Measurements* inside the page *ZigBee Monitor*.

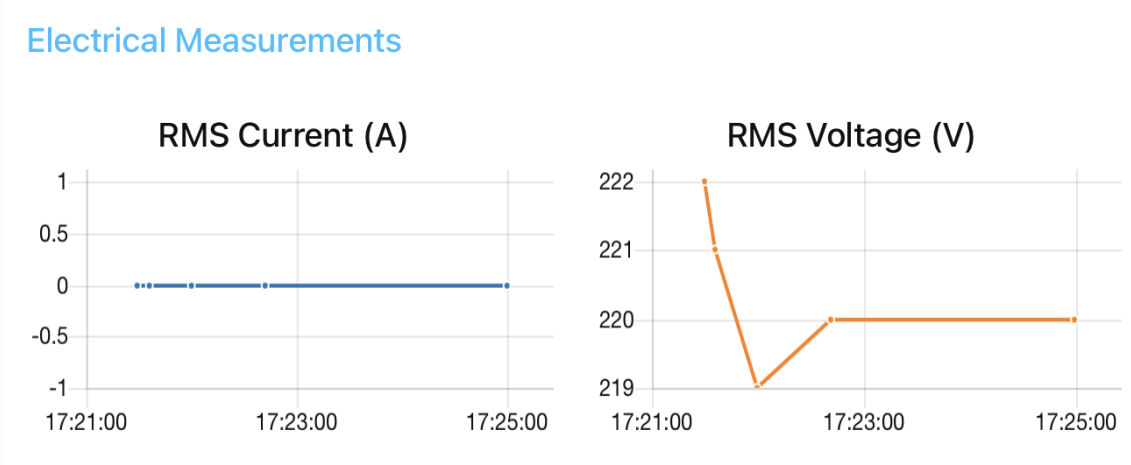


Figure 4: *Node-Red dashboard showing real-time RMS Current (left) and RMS Voltage (right) charts.*

5 Task 3 – Link Status Handling and ThingSpeak Publishing (2 points)

5.1 Description

When the packet retrieved from `challenge3.csv` at index `N` is a Link Status packet (i.e., the command string contains `Link Status (0x08)`), the flow extracts the outgoing cost for each link associated with the ZigBee source address. If a later Link Status packet arrives from the same source address, the stored costs are updated. At the end of the execution (after 200 ID messages have been processed), all collected link costs are saved to `outgoing_cost.csv` with the format `No., Source, Destination, Cost`.

Subsequently, the smallest source address in `outgoing_cost.csv` (treating addresses as hexadecimal integers) is identified. Its destination addresses are sorted in ascending hexadecimal order, and the corresponding outgoing costs are sent one by one to a public ThingSpeak channel via the HTTP API, placing each cost value in `field1`. Messages are sent in destination-address order with a rate limit of one message every 20 seconds.

5.2 Implementation

This branch processes incoming MQTT messages to detect Link Status packets, maintains a running table of outgoing costs in flow context, and at the 200-message limit dumps the results to a CSV file and uploads them to ThingSpeak.

Node 1 — Process Link Status

This function node extracts the outgoing cost for every neighbour listed in the Link Status packet and stores the results in a flow-context object (`flow.costTable`) keyed by ZigBee source address. If a later packet arrives from the same source address, its entries overwrite the previously stored values, implementing the update semantics required by the specification.

Because the `Command String` field uses Python-style single-quoted dictionaries rather than valid JSON, the node uses a regular expression to extract each link entry without attempting

a full JSON parse.

Listing 11: *Process Link Status*

```
const packetType = msg.row['Packet Type'] || '';

if (!packetType.includes('Link Status (0x08)')) {
    return null;
}

let costTable = flow.get('costTable') || {};

const source = msg.row['Source Address ZigBee'].trim();
const cmdStr = msg.row['Command String'];

// Extract link entries with a regex (command string is Python-like,
// not valid JSON, so JSON.parse cannot be used directly)
const linkRegex =
    /\{'Address':\s*'(0x[0-9a-fA-F]+)',\s*'Incoming Cost':\s*'\d+',\s*'Outgoing Cost'
        :\s*'(\d+)'\}/g;

let match;
costTable[source] = costTable[source] || {};

while ((match = linkRegex.exec(cmdStr)) !== null) {
    const address = match[1];
    const outgoingCost = match[2];
    costTable[source][address] = outgoingCost;
}

flow.set('costTable', costTable);

return null;
```

Connections: no output (returns `null` always); the cost table is updated in flow context as a side effect.

Node 2 — Generate outgoing_cost.csv

Triggered by the second output of *Counter + ID* when the 200-message limit is reached. It reads `flow.costTable`, serialises every source–destination–cost triple into CSV rows, and sets `msg.payload` to the full file content (including the header line).

Listing 12: *Generate outgoing_cost.csv*

```
const costTable = flow.get('costTable') || {};

let rows = ['No.,Source,Destination,Cost'];
let rowNum = 1;

const sources = Object.keys(costTable);
sources.forEach(source => {
    const destinations = Object.keys(costTable[source]);
    destinations.forEach(dest => {
        const cost = costTable[source][dest];
        rows.push(`${rowNum},${source},${dest},${cost}`);
        rowNum++;
    });
});
```

```
msg.payload = rows.join('\n');
return msg;
```

Connections: output → *Save outgoing_cost.csv* and → *Prepare ThingSpeak Messages*.

Node 3 — Save outgoing_cost.csv

A built-in *write file* node configured to overwrite the file at the specified path with the content of `msg.payload`. The newline option is disabled because line endings are already embedded in the payload string.

Connections: no further output.

Node 4 — Prepare ThingSpeak Messages

Reads `flow.costTable`, finds the source address with the smallest hexadecimal value, sorts its destination addresses in ascending hexadecimal order, and stores the ordered list of {cost, destination, source} objects in `flow.thingspeakMessages`. It then emits the first message immediately.

Listing 13: *Prepare ThingSpeak Messages*

```
const costTable = flow.get('costTable') || {};

// Find the source with the smallest hex address
const sources = Object.keys(costTable);
const smallestSource = sources.sort((a, b) =>
    parseInt(a, 16) - parseInt(b, 16)
)[0];

// Sort destinations in ascending hex order
const destinations = Object.keys(costTable[smallestSource]);
destinations.sort((a, b) => parseInt(a, 16) - parseInt(b, 16));

const messages = destinations.map(dest => ({
    cost: costTable[smallestSource][dest],
    destination: dest,
    source: smallestSource
}));

flow.set('thingspeakMessages', messages);
flow.set('thingspeakIndex', 0);

msg.payload = messages[0];
return msg;
```

Connections: output → *Format HTTP ThingSpeak*.

Node 5 — Format HTTP ThingSpeak

Builds the ThingSpeak HTTP GET URL by injecting the outgoing cost into the `field1` query parameter together with the channel Write API Key.

Listing 14: *Format HTTP ThingSpeak*

```
const WRITE_API_KEY = 'OUR_API_KEY';
const cost = msg.payload.cost;
```

```
msg.url = `https://api.thingspeak.com/update?api_key=${WRITE_API_KEY}&field1=${cost}`
    ;
msg.method = 'GET';

return msg;
```

Connections: output → *HTTP Request*.

Node 6 — HTTP Request

A built-in *http request* node configured to perform a GET request to the URL stored in `msg.url`. The response is returned as a UTF-8 string. A successful update returns the entry ID assigned by ThingSpeak.

Connections: output → *Next ThingSpeak Message*.

Node 7 — Next ThingSpeak Message

After each successful HTTP request this node increments the index into the message list and emits the next message, or stops the loop when all destinations have been uploaded.

Listing 15: *Next ThingSpeak Message*

```
const messages = flow.get('thingspeakMessages') || [];
let index = flow.get('thingspeakIndex') || 0;
index++;
flow.set('thingspeakIndex', index);

if (index >= messages.length) {
    node.warn('ThingSpeak: all messages sent!');
    return null;
}

msg.payload = messages[index];
return msg;
```

Connections: output → *Delay 20s*.

Node 8 — Delay 20s

A built-in *delay* node set to delay each message by exactly 20 seconds, enforcing the rate limit of one ThingSpeak update per 20 seconds required by the specification.

Connections: output → *Format HTTP ThingSpeak* (closes the loop).

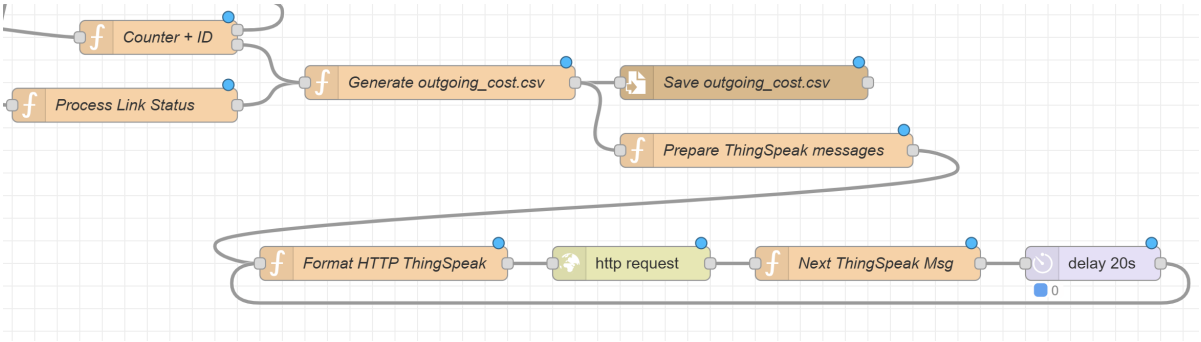


Figure 5: *Link Status branch flow.*

5.3 Output: `outgoing_cost.csv`

Listing 16: *Excerpt of generated* `outgoing_cost.csv`.

```
No.,Source,Destination,Cost
1,0x1cd8,0x0000,1
2,0x1cd8,0x0112,3
3,0x1cd8,0x1e15,1
4,0x1cd8,0x3d95,1
5,0x1cd8,0x4e11,3
```

5.4 ThingSpeak Channel

https://thingspeak.mathworks.com/channels/3378810

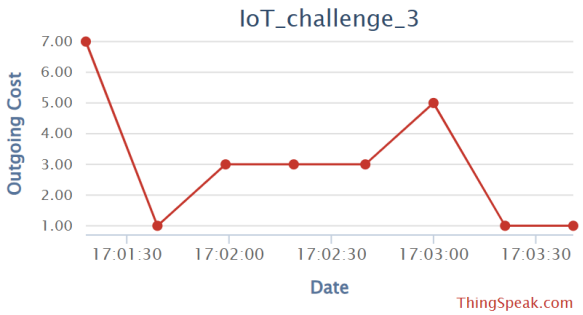


Figure 6: *Outgoing costs in ThingSpeak.*